SOCKET-SCTP-WEBSOCKET

소켓·WebSocket·SCTP

연관 기출 · 131-2-5, 132-3-4

한눈에 이해

소켓은 프로그램이 네트워크를 통해 데이터를 보내고 받는 인터페이스입니다. TCP·UDP·SCTP는 그 아래에서 전송을 담당합니다.

정의와 핵심 원리

소켓은 주소·포트 등의 종단점을 통해 데이터를 주고받는 운영체제 인터페이스입니다. RFC 6455의 WebSocket은 HTTP 오프닝 핸드셰이크 후 TCP 연결에서 메시지 프레임을 교환합니다. SCTP는 메시지 단위의 신뢰성 있는 전송을 제공합니다.

• TCP 서버는 socket → bind → listen → accept 후 연결된 소켓으로 송수신하고 클라이언트는 socket → connect를 수행합니다. 스트림의 read/write 한 번이 애플리케이션 메시지 하나와 일치하지 않으므로 길이·구분자 등 프레이밍이 필요합니다.

• UDP는 연결 설정 없이 데이터그램을 교환합니다. 손실·순서·혼잡·MTU는 응용에서 고려합니다. HTTP는 요청과 응답의 의미를 정의하는 응용 프로토콜이므로 소켓과 같은 층에서 비교하지 않습니다. WebSocket을 배포할 때도 인증·인가·Origin 검사·TLS를 검토합니다.

• SCTP는 association을 만들고 공통 헤더와 chunk를 사용합니다. INIT → INIT ACK → COOKIE ECHO → COOKIE ACK의 쿠키 방식, 스트림별 순서화, TSN·SACK, 멀티호밍을 제공합니다. 멀티스트리밍이 모든 종류의 HOL 지연을 없애거나 멀티호밍이 자동 대역 합산을 뜻하지는 않습니다.

구조·비교·판단

구분	핵심 원리	적용·한계
TCP socket	신뢰성 바이트 스트림	응용 프레이밍 필요
WebSocket	양방향 메시지 프레임	브라우저·웹 연계
SCTP	멀티스트림 메시지·멀티호밍	지원망·방화벽·구현 검토

적용 예제

실시간 대시보드

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 서버가 초당 상태 변경을 브라우저로 전달

1. 처음 HTTP 오프닝에서 사용자와 Origin을 확인합니다.
2. WebSocket 세션에서 구독 권한을 검증하고 메시지 크기를 제한합니다.
3. 연결이 끊기면 마지막 이벤트 번호를 기준으로 재동기화합니다.
4. 일반 조회는 HTTP, 실시간 푸시는 WebSocket을 써서 상태를 관리합니다.

결과·해석: 소켓 API·전송 계층·응용 프로토콜의 역할을 구분할 수 있습니다.

전제·주의: WebSocket 연결이 살아 있어도 업무 이벤트의 exactly-once 처리는 별도 설계입니다.

답안 골격과 혼동 방지

계층과 정의 → 소켓 호출 흐름 → TCP·UDP·WebSocket 비교 → SCTP 구조·쿠키 → 보안·복구

- HTTP를 소켓을 사용하지 않는 통신이라고 설명하지 않습니다.
- SCTP의 association은 여러 스트림과 경로를 다룰 수 있습니다. TCP 연결과 비교할 때 이 차이를 설명합니다.

내용을 가리고 회상

1. TCP에서 메시지를 받을 때 프레이밍이 필요한 이유는 무엇입니까?
2. SCTP의 스트림과 TSN은 어떤 역할을 합니까?
3. WebSocket 연결을 맺은 뒤에도 구독 권한을 확인해야 하는 이유는 무엇입니까?

적용 기준과 근거

명시한 출처 판본의 원리를 설명하며 제품별 구현 차이는 별도 확인합니다.

내용 검토 상태: 출처 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- RFC9260: Stream Control Transmission Protocol - R. Stewart et al. / IETF; 2022-06; 확인 2026-11-05.
- RFC6455: The WebSocket Protocol - I. Fette; A. Melnikov / IETF; 2011-12; 확인 2026-11-05.
- RFC9293: Transmission Control Protocol - W. Eddy (ed.) / IETF; 2022-08; 확인 2026-11-05.